

Lecture: 0010

Logic Minimisation & Universal Gates

How engineers take 16 gates and leave only 2 — the art of doing less.

CSA222: Modern Computer Architecture

Where We Left Off — Recap of L1

Four gates, a handful of laws. Today we turn them into minimal circuits.



AND • OR • NOT • XOR

The four core gates and their truth tables — the vocabulary of all digital logic.



Boolean values 1 / 0

Every signal is a voltage that means TRUE or FALSE — nothing in between.



Identity • Null • Idempotent

$A+0=A$, $A \cdot 1=A$, $A+1=1$, $A+A=A$ — the algebra that simplifies expressions.



NAND / NOR previewed

We hinted these are 'universal'. Today we prove it and build with them.

Prereq check: if truth tables for AND/OR/NOT/XOR feel automatic, you're ready.

Today's Roadmap

Three stops, one destination: fewer gates, same truth table.

1

Truth Tables → Expressions

SOP & POS: two mirror-image recipes for turning any truth table into algebra.

2

Karnaugh Maps

Minimisation becomes a visual grouping game — no messy algebra required.

3

Universal Gates

DeMorgan's theorems prove one gate — NAND — can build every circuit that exists.

Why Shrink a Circuit?

Every extra gate is silicon area, power burned, and picoseconds lost.

16 → 2

A typical 4-variable function

can collapse from 16 gates
to as few as 2 — same
truth table, 8× less silicon.



AREA

Fewer gates → smaller die →
more chips per wafer →
cheaper.



POWER

Less switching → lower dynamic
power → longer battery life.



SPEED

Shorter gate chains → faster
propagation → higher clock.

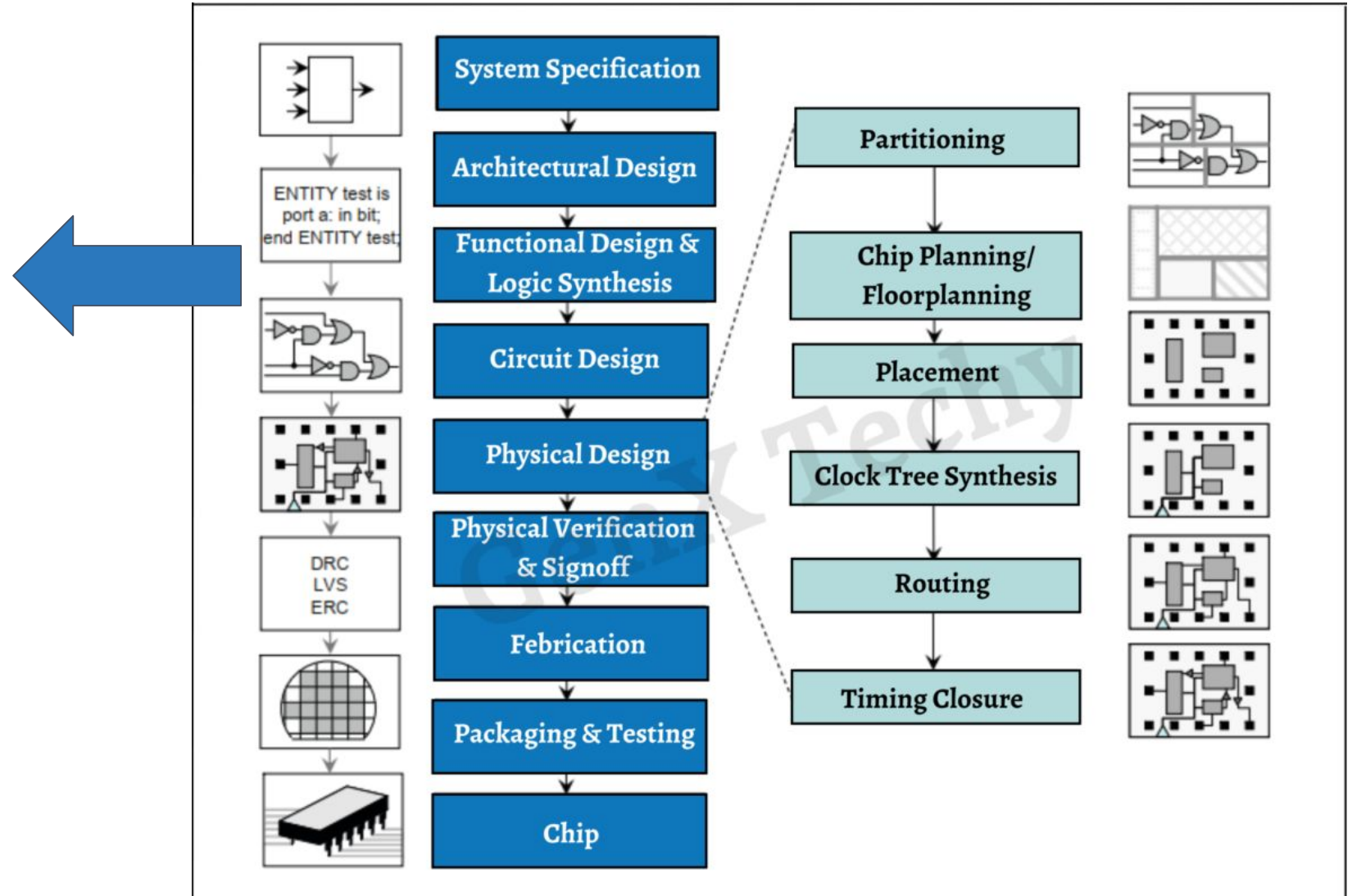


COST

Less silicon + higher yield =
more profit at fab scale.

VLSI Design Flow (Logic Synthesis)

- This is our Goal to understand !!
- Apple's M3 packs 98 billion transistors on TSMC's second-generation 3nm process (N3E) — at that density, a 1% gate-count reduction, multiplied across billions of gates, is real die area and real dollars.
- This is exactly why chip companies run logic synthesis tools (Synopsys Design Compiler, Cadence Genus) before fabrication.



From Truth Table to Expression

You can't minimise a table. First turn it into algebra — two canonical ways.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Highlighted rows are where $F = 1$

SOP — Sum of Products

OR together the rows where $F = 1$. Each such row $\rightarrow$ one AND term (a minterm).

$$F = A'B'C + A'BC + AB'C + ABC$$

POS — Product of Sums

AND together the rows where $F = 0$. Each such row $\rightarrow$ one OR term (a maxterm).

$$F = (A+B+C)(A+B'+C)(A'+B+C)(A'+B'+C)$$

Same function, two equivalent descriptions — pick whichever gives fewer terms.

Sum of Products (SOP)

For every row where $F = 1$, write a minterm. Then OR them together.

1

Find every row where $F = 1$

These are the only rows that contribute to an SOP expression.

2

Write one minterm per row

A single AND term true for exactly that one input combination.

3

Variable = 1 $\rightarrow X$ • Variable = 0 $\rightarrow X'$

Keep it if it's 1; invert it if it's 0.

4

OR all minterms together

The finished canonical SOP expression.

WORKED EXAMPLE

Row 5: $A=1, B=0, C=1, F=1$

$A = 1$

$\rightarrow A$

$B = 0$

$\rightarrow B'$

$C = 1$

$\rightarrow C$

minterm $m_5 = A \cdot B' \cdot C$

Product of Sums (POS)

The mirror image — for every 0-row write a maxterm, then AND them.

1

Find every row where $F = 0$

The 0-rows are the ones that build a POS expression.

2

Write one maxterm per row

A single OR term that is 0 for exactly that one combination.

3

Variable = 1 $\rightarrow X'$ • Variable = 0 $\rightarrow X$

Inverted relative to SOP — this is THE trap. Flip it.

4

AND all maxterms together

The finished canonical POS expression.

WORKED EXAMPLE

Row 2: $A=0, B=1, C=0, F=0$

$A = 0$

$\rightarrow A$

$B = 1$

$\rightarrow B'$

$C = 0$

$\rightarrow C$

maxterm M2 = $(A + B' + C)$

Minterm & Maxterm Shorthand

Textbooks and exams write canonical forms in one compact line — read it instantly.

Σ NOTATION

Sigma means SOP — OR together the listed minterm numbers.

$$F = \Sigma m(1, 3, 5, 7)$$

Π NOTATION

Pi means POS — AND together the listed maxterm numbers.

$$F = \Pi M(0, 2, 4, 6)$$

WHY IT MATTERS

- The number in $\Sigma m(\dots)$ is the decimal value of that row's input bits.
- Row $A=1, B=0, C=1 \rightarrow$ binary $101 \rightarrow 5$, so it appears as $m5$.
- Σ and Π over the SAME function are complementary: every index is in exactly one.

Boolean Algebra: Core Laws

The algebraic toolkit K-maps are about to make visual — no grid required.

LAW FAMILY	KEY IDENTITIES	WHY IT MATTERS
Identity & Null	$A+0=A$ $A\cdot 1=A$ $A+1=1$ $A\cdot 0=0$	A single wire or a constant wins — no gate needed.
Idempotent & Complement	$A+A=A$ $A\cdot A=A$ $A+A'=1$ $A\cdot A'=0$	No duplicate gates; a signal and its inverse cancel.
Absorption	$A+A\cdot B=A$ $A\cdot (A+B)=A$	The bigger term swallows the smaller one whole.
Distributive	$A(B+C)=AB+AC$ $A+BC=(A+B)(A+C)$	Yes — OR distributes over AND too, unlike ordinary algebra.

Every K-map grouping you draw next is one of these laws applied visually instead of algebraically.

Karnaugh Maps

Minimisation as pattern recognition — group adjacent 1s and watch variables cancel.

Enter Karnaugh Maps a' b' edit

A truth table re-organised so neighbours differ by exactly ONE bit.

A\B	0	1
0	0	1
1	0	1

Group the B=1 column $\rightarrow F = B$

A cancels: both $A=0$ and $A=1$ give 1.

THE KEY INSIGHT

- **Each cell = one minterm.**
The grid is just the truth table, rearranged.
- **Adjacent 1-cells differ by one variable.**
So grouping them makes that variable cancel.
- **Grouping = simplification.**
Bigger groups $\rightarrow$ more variables cancel $\rightarrow$ shorter expression.
- **It's pattern recognition, not algebra.**
Your eye does the work the Boolean laws would.

3-Variable K-Map

8 minterms in Gray-code order — every horizontal step flips exactly one bit.

A\BC	00	01	11	10
0	0	1	1	0
1	0	1	1	0

Group of 4 → **F = C**

4 minterms collapse into a single variable.

GRAY-CODE ORDERING

Columns run $00 \rightarrow 01 \rightarrow 11 \rightarrow 10$, not binary count. This guarantees every step flips just one variable, keeping adjacency meaningful.

LEGAL GROUP SIZES

1 cell → full 3-variable minterm

2 cells

→ 2-variable term

4 cells

→ 1-variable term

8 cells

→ constant 1

4-Variable K-Map

16 cells, both axes Gray-coded. Bigger groups = bigger savings.

AB\CD	00	01	11	10
00	1	0	0	1
01	1	1	1	1
11	0	1	1	0
10	0	0	0	0

GROUP ANATOMY

OCTET

8 cells

3 variables cancel

QUAD

4 cells

2 variables cancel

PAIR

2 cells

1 variable cancels

SINGLE

1 cell

nothing cancels

Each doubling of group size cancels exactly one more variable.

$$F = A'B + BD + A'D'$$

The Wrap-Around Trick

A K-map is a torus (donut) — opposite edges and all four corners are neighbours.

AB\CD	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	0	0	0
10	1	0	0	1

THINK: DONUT

What looks far apart on flat paper is actually adjacent. Opposite edges are glued together.

- Left column ↔ right column
- Top row ↔ bottom row
- All four corners = one group

4 corners → one quad → $F = B'D'$

The Don't-Care (X)

Some input combinations never happen — treat their output as whatever helps.

AB\CD	00	01	11	10
00	1	1	X	0
01	0	X	X	0
11	0	0	0	0
10	0	0	0	0

Here the X's become 1 → one big green quad.

WHERE DO X's COME FROM?

Impossible inputs

A BCD digit decoder never sees codes 10–15.

Unconnected outputs

A result bit nobody reads downstream.

Guaranteed by system

A sensor that physically can't produce that combo.

RULE Assign each X to 0 or 1 — whichever makes your groups BIGGER.

Full Worked Example: Table → Circuit

One function, start to finish — and it beats the 16→2 promise.

AB\CD	00	01	11	10
00	1	1	0	0
01	1	1	0	0
11	1	1	0	0
10	1	1	0	0

$F = C'$

NAIVE SOP

9 gates

8 four-input ANDs + one big eight-input OR

MINIMISED

1 gate

$F = B'$ — a single NOT gate does it all

CANCELLED

3 of 4 vars

A, C, and D all vanish from the answer

CALLBACK

Beats 16→2

Slide 4's opening promise — beaten

This is exactly what a logic-synthesis tool (Synopsys, Cadence) does — automatically, at billion-gate scale, in seconds.

DeMorgan & Universal Gates

One gate, wired to itself, builds every Boolean function — and every chip you own.

DeMorgan's Theorems

Two rules that let you swap freely between AND-world and OR-world.

RULE 1

$$\sim(A \cdot B) = \sim A + \sim B$$

$$\text{NOT}(A \text{ AND } B) = (\text{NOT } A) \text{ OR } (\text{NOT } B)$$

RULE 2

$$\sim(A + B) = \sim A \cdot \sim B$$

$$\text{NOT}(A \text{ OR } B) = (\text{NOT } A) \text{ AND } (\text{NOT } B)$$

The one-line memory hook: ***“Break the bar, flip the sign.”***

Bubble Pushing

DeMorgan as a physical motion — slide a bubble through a gate and it flips type.

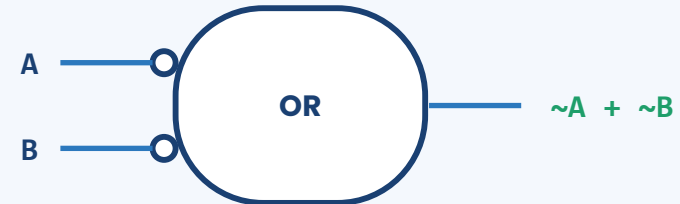
BEFORE



AND gate with an output bubble = NAND.



AFTER



OR gate with input bubbles — same function!

Bubble pushing IS DeMorgan — just drawn instead of written. Same theorem, no new rule.

DeMorgan in Practice

Apply it repeatedly, one bar at a time, from the outside in.

GOAL

Simplify: $\sim(A \cdot B + C)$

STEP 1

Break the outer NOT over the OR: $\sim(A \cdot B) \cdot \sim C$

STEP 2

Break the remaining bar on $\sim(A \cdot B)$: $(\sim A + \sim B) \cdot \sim C$

DONE

$(\sim A + \sim B) \cdot \sim C$

Scan for EVERY remaining bar after each step — students stop after one and miss the nested NOT.

Quick Check

Show of hands — no wrong answers, only data points.

Q. $\sim(A + B)$ is equivalent to...

A $\sim A + \sim B$

B $\sim A \cdot \sim B$

C $A \cdot B$

D $A + B$

✓ *DeMorgan's Rule 2*

NOT of an OR becomes an AND of NOTs — the theorem we prove on the next slide.

NAND Is Everything

One gate type, wired to itself the right way, builds every Boolean function.

ONLY
NAND

is needed to build every gate — and
therefore every CPU.

THE PROOF, IN THREE STEPS

NOT

= $\text{NAND}(A, A)$

Tie both inputs together.

AND

= NAND then NOT

Invert the NAND's output.

OR

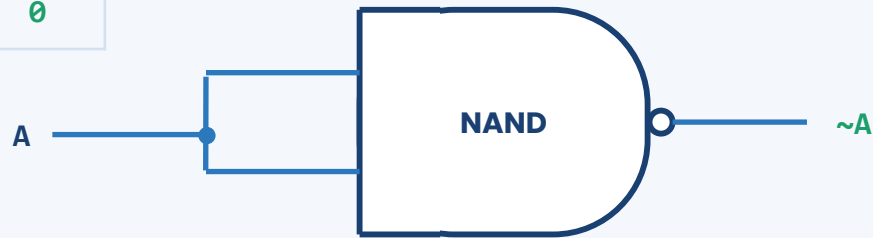
= NAND of inverted inputs

DeMorgan, realised in silicon.

Build #1 — NOT from NAND

Tie both inputs of a NAND together. Done.

A	$\sim A$
0	1
1	0



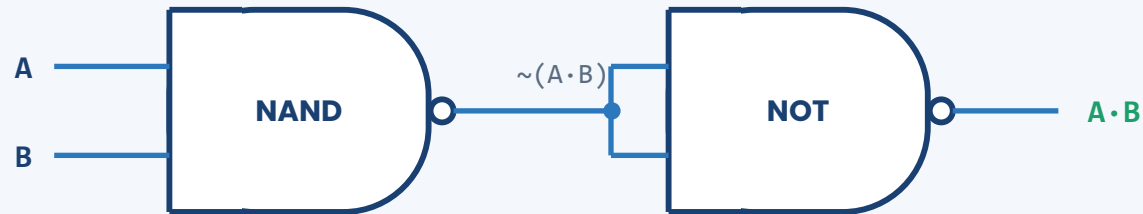
RESULT

$$\text{NOT}(A) = \text{NAND}(A, A)$$

With both inputs equal, NAND is 1 when $A=0$ and 0 when $A=1$ — precisely the NOT truth table. One gate, zero extra parts.

Build #2 — AND from NAND

A NAND, followed by another NAND wired as a NOT. Two gates.



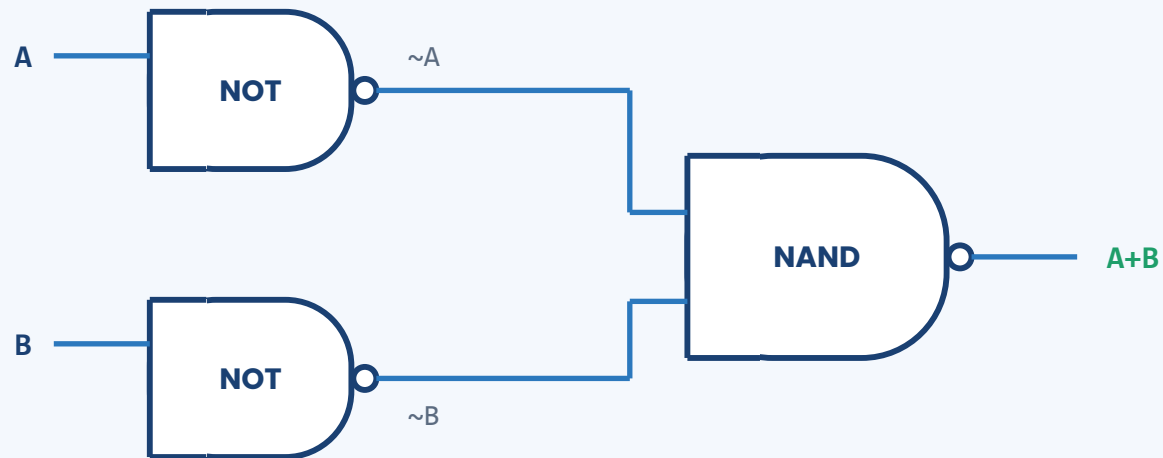
RESULT

$$\text{AND}(A, B) = \text{NAND}(\text{NAND}(A, B), \text{NAND}(A, B))$$

The first NAND gives $\sim(A \cdot B)$; the second — wired as a NOT — inverts it back to $A \cdot B$. This is why a 'real' AND cell costs extra transistors.

Build #3 — OR from NAND

Invert each input, then NAND the results. DeMorgan in silicon.



RESULT

$$\text{OR}(A, B) = \text{NAND}(\text{NAND}(A, A), \text{NAND}(B, B))$$

$\text{NAND}(\sim A, \sim B) = \sim(\sim A \cdot \sim B) = A + B$ by DeMorgan. This is Slide 20's Rule 1 turned into a working circuit — algebra becoming hardware.

NOR Is Universal Too

Every NAND trick has a NOR mirror — swap AND $\leftrightarrow$ OR and the inversion positions.

BUILD	NAND-ONLY	NOR-ONLY
NOT	<code>NAND(A, A)</code>	<code>NOR(A, A)</code>
AND	<code>NAND(NAND(A,B), NAND(A,B))</code>	<code>NOR(NOR(A,A), NOR(B,B))</code>
OR	<code>NAND(NAND(A,A), NAND(B,B))</code>	<code>NOR(NOR(A,B), NOR(A,B))</code>

Both are universal — but NAND maps more naturally to CMOS transistor layouts. *That's why it wins (next slide).*

Bonus: NAND flash vs NOR flash are named after which gate topology their memory cells resemble.

Why Chip Fabs Pick NAND

At the transistor level, a CMOS NAND needs fewer transistors than an AND.

CMOS NAND

4

TRANSISTORS

2 NMOS + 2 PMOS — a direct, natural mapping.

CMOS AND

6

TRANSISTORS

NAND + an inverter (adds 2 more transistors).

Multiply by billions of gates per chip. Apple's M4 packs 28 billion transistors on TSMC's 3nm — *that 2-transistor delta becomes a city block of silicon.*

Common Mistakes to Avoid

The exact slips that cost marks in exams and interviews.



Flipping the SOP/POS inversion rule

SOP: $1 \rightarrow X$, $0 \rightarrow X'$. POS is the opposite. Say it out loud each time.



Forgetting wrap-around & corners

Always check edges and all four corners before finalising groups.



Reading K-map columns as binary

They're Gray code (00,01,11,10). Binary order breaks single-bit adjacency.



Stopping DeMorgan after one bar

Scan for every remaining NOT bar — nested ones need another pass.



Grouping diagonally

Diagonal cells differ by two bits — never adjacent. Only rectangles of size 2^n .



Thinking NAND is 'more powerful'

It's self-sufficient, not stronger. AND+OR+NOT together are equally universal.

Key Takeaways

Five sentences. Pin them up — everything else in this course builds from here.

- 01 Any truth table → canonical SOP (from the 1-rows) or POS (from the 0-rows).
- 02 K-maps turn minimisation into a visual pattern-finding game with Gray-code adjacency.
- 03 Bigger legal groups cancel more variables; the grid wraps like a torus, and don't-cares are free wildcards.
- 04 DeMorgan: break the bar, flip the sign — the bridge between AND-world and OR-world.
- 05 NAND (or NOR) alone can implement every Boolean function — and fewer CMOS transistors is why fabs choose it.

Practice & Homework

Three activities plus exit problems — pair up on the whiteboard.

01

Truth Table → SOP

3-input table with 1s at rows {1,3,5,7}.
Write the canonical SOP, then
minimise on a K-map.

02

K-Map Race

In pairs, minimise three 4-variable
K-maps under a timer. Compare who
found the largest groups.

03

NAND-Only Challenge

Implement AND, OR, NOT — then XOR
— using only 2-input NAND gates.
Verify with a truth table.

EXIT TICKET

1. Minimise $F(A,B,C,D) = \sum m(0,2,5,7,8,10,13,15)$ using a K-map.
2. Apply DeMorgan to $\sim(A \cdot B + C)$.
3. Build a 2-input OR gate using only NAND gates — sketch and verify.

END OF L02

One gate, every circuit.

NEXT — L03

Combinational Building Blocks: MUX, DEMUX, Decoders & Adders — the actual guts of a CPU's ALU, built from everything you minimised today.



**Thanks
for
watching!**